Lesson 6: 3D Objects

Lesson 6: 3D Objects 1
6.1 Surface Representations 2
6.2 Planes 3
6.2.1 Planes 3
6.3 Surface Tangents and Normals 3
6.4 Parametric Surfaces 3
6.5 3D Affine Transformations 4
6.6 Spherical Coordinates 4
6.7 Nonlinear Transformations 5
6.8 Representing Triangle Meshes 5
6.9 Generating Triangle Meshes 5
Lesson 6 Discussion Session 5

6.1 Surface Representations

As with 2D objects, we can represent 3D objects in **parametric** and **implicit** forms. (There are also explicit forms for 3D surfaces — sometimes called "height fields"

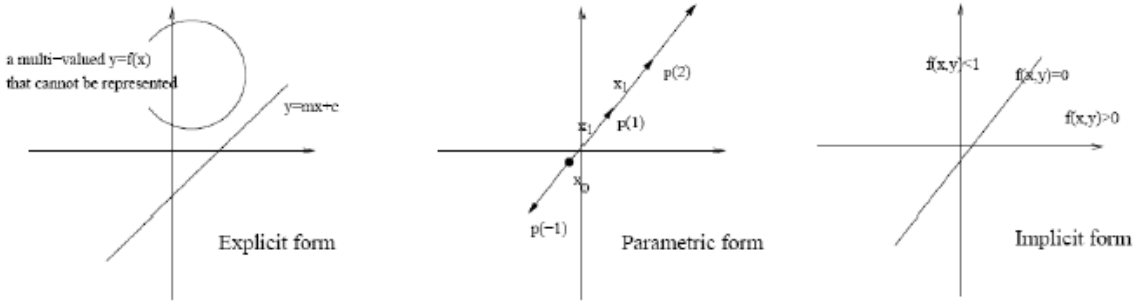


Figure 6.1: The explicit, parametric and implicit forms of a line.

The **explicit** form of representing a line is y=mx+c. this form is limited to straight line alone observe the above diagram.

So suppose we want to model a line, any line, in our Computer Graphics system. An alternative way to do this is to using the **parametric form**:

$p(s) = x_0 + sx_1$

Here vector x_0 indicates the start point of the line, and x_1 a vector in the positive direction of the line. We have introduced the dummy parameter s as a mechanism for iterating along the line; given an s we obtain a point p(s) some distance along the line. When s = 0 we are at the start of the line; positive values of s move us forward along the line and negative values of s move us backward (Figure 6.1, middle).

By convention we use p(0) to denote the start of a space curve, and p(1) to denote the end point of the curve. That is, we increase s from 0 to 1 to move along the entire, finite length of the curve.

We can generalize this concept to produce curved trajectories. Consider p(s) as the position of a particle flying through space, and s as analogous to time. Then x_0 is the start position of the particle, and x_1 is the velocity of the particle. We can add a term x_2 for acceleration as follows:

$p(s) = x_0 + sx_1 + s^2x_2$

to yield a quadratic curve. We can continue adding terms to any order n:

$$\underline{p}(s) = \sum_{i=1}^{n} s^i \underline{x}_i$$

although in practice it is unusual in Computer Graphics to use anything above n = 3 (cubic curves

For example a more convenient parameterization of a circle might be:

$$\underline{p}(\theta) = \begin{pmatrix} r\cos\theta \\ r\sin\theta \end{pmatrix}$$

The final representation of space curve that we will look at is the **implicit form**. In **implicit form** we express the equation of a curve as a function of all of its coordinates equal to a constant (often zero), e.g. in 2D we have f(x, y) = 0. We can reach the implicit form for a 2D line easily from the parametric form, by writing out the x and y components of the equation. Here we write x0 = [x0 y0]T and x1 = [u v]T:

$$
\begin{aligned}
x &= x_0 + su \\
y &= y + 0 + sv
\end{aligned}
$$

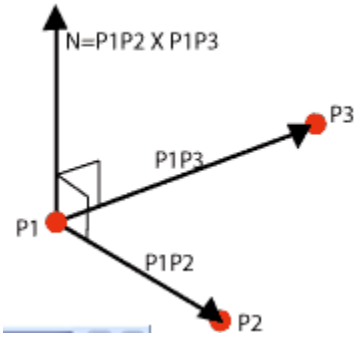
$$
\begin{aligned}
\frac{x - x_0}{u} &= s = \frac{y - y_0}{v} \\
(x - x_0)v &= (y - y_0)u \\
(x - x_0)v - (y - y_0)u &= 0
\end{aligned}
$$

where x_0, y_0, u, and v are constants defining the line. We insert values for a given point (x, y) and if the equation is satisfied (i.e. the left hand side is zero) we are at a point on the line.
Furthermore the term on the left hand side is positive if we are off the line but on one side of it, and negative if we are on the other side. This is because the left hand side evaluates the signed distance from the point (x, y) to the nearest point on the line (Figure 6.1, right).

6.2 Planes

6.2.1 Planes

If we know two vectors that form an angle which is not equal to 0 or 180 degrees, the plane spawn by the two vectors is uniquely defined. This means that if we know three points that are not on a line, the plane is uniquely defined. The **plane's normal** is defined by the **two vectors cross product**.



The general form for a plane is Ax+By+Cz+D=0
where the vector |A B C| is the plane's normal. We will not prove or deduce or give reasons for this. Notice that we use the right hand rule to calculate the normal. This is important when we are stating normals in OpenGL. Since OpenGL separates between front and back on surfaces, it is important that we state normals in a systematic way. D can be found by inserting a point in the plane in the general plane equation.

6.3 Surface Tangents and Normals

The **tangent** to a curve at ¯p is the instantaneous direction of the curve at ¯p.
The **tangent plane** to a surface at ¯p is analogous. It is defined as the plane containing tangent vectors to all curves on the surface that go through ¯p. The **Normal Line** at is the **line** that passes through and is perpendicular to the tangent plane at and perpendicular to the **surface** at.

6.4 Parametric Surfaces

A **parametric surface** is a surface in the **Euclidean space** which is defined by a parametric equation with two parameters. Parametric representation is a very general way to specify a surface, as well as **implicit**

representation. Surfaces that occur in two of the main theorems of **vector calculus, Stokes' theorem** and the **divergence theorem**, are frequently given in a parametric form.

- The simplest type of parametric surfaces is given by the graphs of functions of two variables:

$$z = f(x, y), \quad \vec{r}(x, y) = (x, y, f(x, y)).$$

- A rational surface is a surface that admits parameterizations by a rational function. A rational surface is an algebraic surface. Given an algebraic surface, it is commonly easier to decide if it is rational than to compute its rational parameterization, if it exists.
- Surfaces of revolution give another important class of surfaces that can be easily parameterized.
- The straight circular cylinder of radius *R* about *x*-axis has the following parametric representation:

$$\vec{r}(x, \phi) = (x, R\cos\phi, R\sin\phi).$$

6.5 3D Affine Transformations

Three dimensional transformations are used for many different purposes, such as coordinate transforms, shape modeling, animation, and camera modeling.

An affine transform in 3D looks the same as in 2D: $F(\bar{p}) = A\bar{p} + \vec{t}$ for $A \in \mathbb{R}^{3\times3}$, $\bar{p}, \vec{t} \in \mathbb{R}^3$. A homogeneous affine transformation is

$$\hat{F}(\hat{p}) = \hat{M}\hat{p}, \text{ where } \hat{p} = \begin{bmatrix} \bar{p} \\ 1 \end{bmatrix}, \hat{M} = \begin{bmatrix} A & \vec{t} \\ \vec{0}^T & 1 \end{bmatrix}.$$

Translation: $A = I$, $\vec{t} = (t_x, t_y, t_z)$.

Scaling: $A = \text{diag}(s_x, s_y, s_z)$, $\vec{t} = \vec{0}$.

Rotation: $A = R$, $\vec{t} = \vec{0}$, and $\det(R) = 1$.

3D rotations are much more complex than 2D rotations, so we will consider only elementary rotations about the x, y, and z axes.

For a rotation about the z-axis, the z coordinate remains unchanged, and the rotation occurs in the x-y plane. So if $\bar{q} = R\bar{p}$, then $q_z = p_z$. That is,

$$\begin{bmatrix} q_x \\ q_y \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} p_x \\ p_y \end{bmatrix}.$$

Including the z coordinate, this becomes

$$R_z(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Similarly, rotation about the x-axis is

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{bmatrix}.$$

For rotation about the y-axis,

$$R_y(\theta) = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{bmatrix}.$$

6.6 Spherical Coordinates

Spherical coordinate system is a coordinate system for three-dimensional space where the position of a point is specified by three numbers: the *radial distance* of that point from a fixed origin, its *polar angle* measured from a fixed zenith direction, and the *azimuthal angle* of its orthogonal projection on a

reference plane that passes through the origin and is orthogonal to the zenith, measured from a fixed reference direction on that plane. It can be seen as the three-dimensional version of the polar coordinate system.

6.7 Nonlinear Transformations

Affine transformations are a first-order model of shape deformation. With affine transformations, scaling and shear are the simplest non-rigid deformations. Common higher-order deformations include tapering, twisting, and bending.

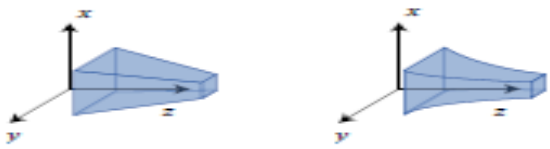
Example:
To create a nonlinear taper, instead of constantly scaling in x and y for all z, as in

$$\bar{q} = \begin{bmatrix} a & 0 & 0 \\ 0 & b & 0 \\ 0 & 0 & 1 \end{bmatrix} \bar{p},$$

let a and b be functions of z, so

$$\bar{q} = \begin{bmatrix} a(\bar{p}_z) & 0 & 0 \\ 0 & b(\bar{p}_z) & 0 \\ 0 & 0 & 1 \end{bmatrix} \bar{p}.$$

A linear taper looks like $a(z) = \alpha_0 + \alpha_1 z$.
A quadratic taper would be $a(z) = \alpha_0 + \alpha_1 z + \alpha_2 z^2$.



(c) Linear taper (d) Nonlinear taper

6.8 Representing Triangle Meshes

A triangle mesh is often represented with a list of vertices and a list of triangle faces. Each vertex consists of three floating point values for the x, y, and z positions, and a face consists of three indices of vertices in the vertex list. Representing a mesh this way reduces memory use, since each vertex needs to be stored once, rather than once for every face it is on; and this gives us connectivity information, since it is possible to determine which faces share a common vertex. This can easily be extended to represent polygons with an arbitrary number of vertices, but any polygon can be decomposed into triangles.

6.9 Generating Triangle Meshes

Mesh generation is the practice of **creating** a **mesh**, a subdivision of a continuous geometric space into discrete geometric and topological cells. **Meshes** are used for rendering to a computer screen and for physical simulation such as finite element analysis or computational fluid dynamics.

Lesson 6 Discussion Session

1. Discuss any three purposes of three dimensional transformations in computer graphics.
2. Implement a rotation function like glRotate in c programming language.